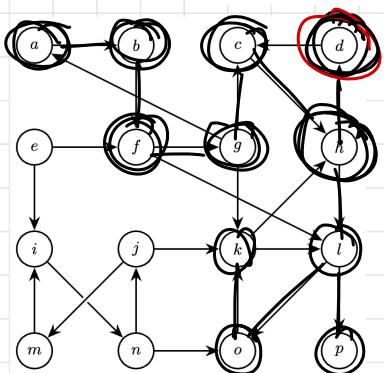
```
DFSALL(G):
    PREPROCESS(G)
    for all vertices v
        unmark v
    for all vertices v
        if v is unmarked
            DFS(v)
```

clock = 0

```
DFS(v):
    mark v
    PREVISIT(v)
    for each edge vw
        if w is unmarked
            parent(w) ← v
            DFS(w)
    POSTVISIT(v)
```

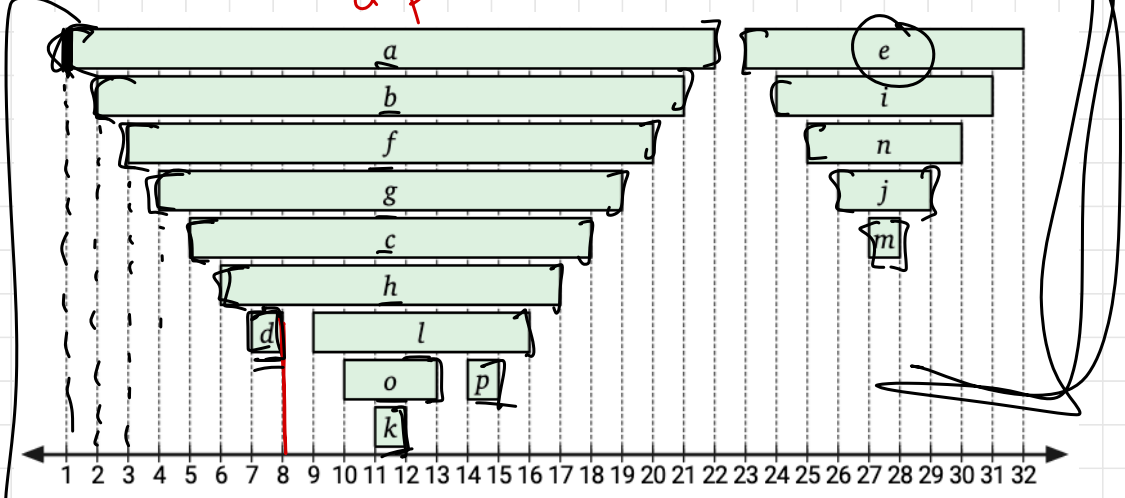
clock++
v.pre = clock

clock++
v.post = clock



d.post = 8

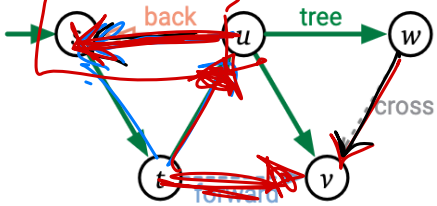
a.pre = 1

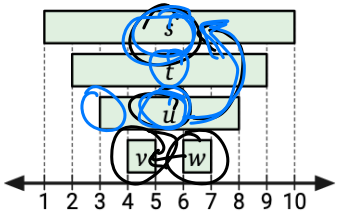


pre: a b f g c h d l o k p e i n j m

post: d k o p l h c g f b a m j n i e

DFS(s) is already going when DFS(u) is called





Suppose $a \to b$ is an edge in G.

① when DFS(a) is called, DFS(b) has not yet been called.

ex $t \to u$ when DFS(t) is called, DFS(u) has not been called yet.
in fact, DFS(t) calls DFS(u)

$t \to v$ when DFS(t) is called, DFS(v) has not been called yet.

Does DFS(t) call DFS(v)?
no
DFS(u) calls DFS(v)

2 types:
tree edges
forward edges

a.pre < v.pre < b.post < a.post

(2) DFS(a) is already going when DFS(b) begins

ex $u \to s$ back edge $\Rightarrow$ cycle



there is a path from b to a in G

$a \to b$ exists in graph,
$a.post < b.post$,
then there is a cycle!

(3) DFS(b) is already finished when DFS(a) begins

cross edge $a \to b$

$b.post < a.pre$

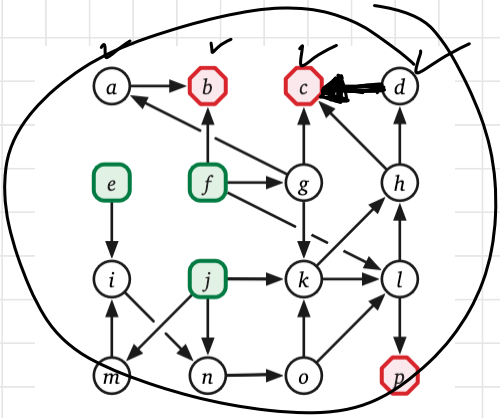
Let G = a graph w/ vertices V, edges E.
A topological order of V is any order $v_1 v_2 \cdots v_n$ so that $i < j$ for all edges $v_i \to v_j$.

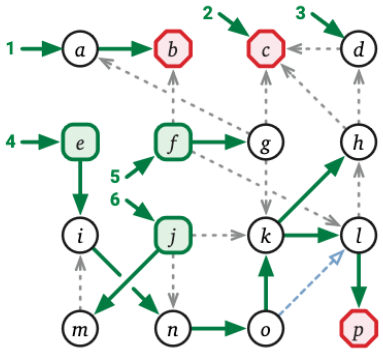
edges always point forward!

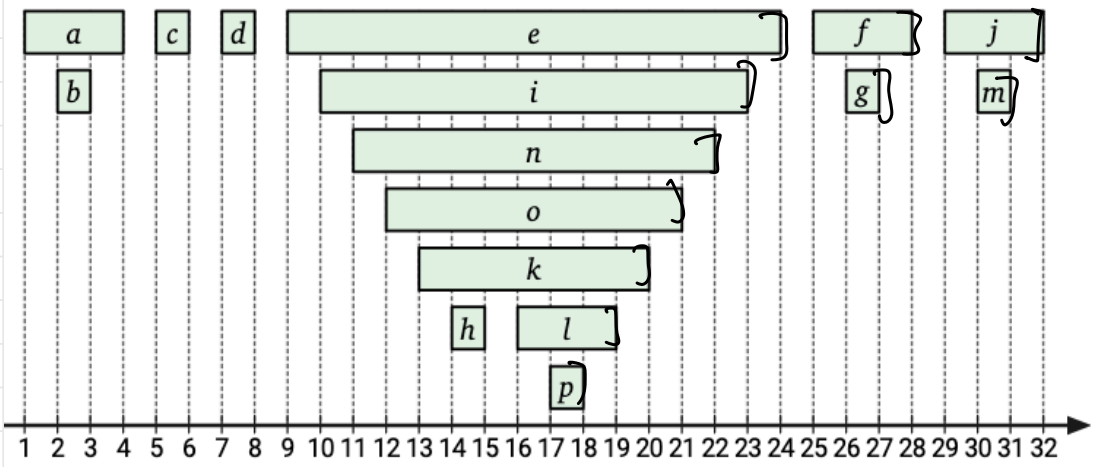
a b c d e f g h i j k l m n o p

v_1 v_2 v_3 v_4

$v_4 \rightarrow v_3$







reverse of a postorder is a topo order